

Behavior Driven Development(BDD)

BDD stands for Behavior Driven Development. It is a testing approach used to verify whether an application behaves as expected from the user's point of view. In BDD, test scenarios are written in simple English using Given, When, and Then statements. This helps developers, testers, and business teams understand the requirements clearly. Cucumber is the most commonly used BDD tool with Selenium.

BDD Structure

BDD scenarios are written using:

- **Given** – Precondition or initial state
- **When** – Action performed by the user
- **Then** – Expected outcome

Advantages of BDD

- Improves communication between business and technical teams.
- Requirements are easy to understand.
- Test cases act as documentation.
- Encourages collaboration.
- Supports test automation.

Common BDD Tools

- Cucumber
- JBehave
- SpecFlow
- Behave (Python)

Test Driven Development vs BDD (Behavior Driven Development)

TDD is a development approach where test cases are written before the actual code to verify functionality

TDD

- TDD focuses on testing the code and its functionality
- Test cases are written before the actual code to verify the functionality of the code.
- Tests are written in a technical format, so they are mainly understood by developers(i.e only technical ppl)
- TDD Mainly used by developers
- TDD Verifies whether the code works correctly.
- Mostly used for unit testing(doubt point)
- Common tools: JUnit, TestNG.

BDD

- BDD focuses on testing the application's behavior according to business requirements and user expectations.
- In BDD, scenarios are written in simple English using Given-When-Then syntax, so developers, testers, and business stakeholders can all understand them.(i.e non technical ppl can also understand)
- Used by developers, testers, and business stakeholders
- Verifies whether the application behaves as expected.
- Mostly used for Acceptance/functional testing(doubt point)
- Common tools: Cucumber, SpecFlow.

Memory Trick

TDD = Test the Code

BDD = Test the Behavior ✓

What is Cucumber?

Cucumber is a **BDD (Behavior Driven Development) tool** used to write and execute test cases in a simple English language format.

It allows developers, testers, and business stakeholders to understand test scenarios easily.

Given, When, Then in Cucumber

Cucumber uses **Gherkin language**, which follows the **Given-When-Then** format:

- **Given** → Precondition (initial setup)
- **When** → Action performed by the user
- **Then** → Expected result

Example

Cucumber uses **Gherkin language**:

Feature: Login Functionality

Scenario: Successful Login

Given the user is on the login page

When the user enters valid credentials

Then the user should be logged in successfully

Why do we use Cucumber?

- Supports BDD approach.
- Scenarios are easy to read and understand.
- Improves communication between technical and non-technical teams.
- Can be integrated with Selenium for automation testing.
- Acts as both documentation and test cases.

Components of Cucumber

1. **Feature File** (.feature) → Contains Given-When-Then scenarios.
2. **Step Definition File** → Java code for each step.
3. **Runner Class** → Executes the test cases.

Notes

"We use **System.setProperty()** to tell Selenium where the browser driver (like ChromeDriver or GeckoDriver) is located." To establish communication between the test script and the browser."

```
System.setProperty("webdriver.chrome.driver", "C:\\Drivers\\chromedriver.exe");
```

```
WebDriver driver = new ChromeDriver();  
driver.get("https://www.google.com");
```

Test Runner class

```
@RunWith(Cucumber.class)  
@CucumberOptions(  
    features = "src/test/resources/features",  
    glue = "stepDefinitions",  
    plugin = {"pretty", "html:target/cucumber-report.html"},  
    monochrome = true  
)
```

- **@RunWith(Cucumber.class)** → Tells JUnit to run Cucumber tests.
- **features** → Path of feature files.
- **glue** → Package containing Step Definition files.
- **plugin** → Generates reports.
- **monochrome = true** → Gives clean console output.

Plugin can be any of these

html:target/report.html Generates HTML report.

json:target/report.json Generates JSON report.

junit:target/report.xml Generates XML report for Jenkins/CI tools.

Parameterization in Cucumber

Parameterization means passing dynamic test data to step definitions instead of hardcoding values.

It helps us execute the same test scenario with different sets of data, improving reusability and reducing code duplication."

Supports data driven testing

Data-Driven Testing (DDT) is a testing approach in which same test case is executed multiple times with different sets of input data.

Instead of hardcoding test data in the script, the data is stored in external sources such as:

- Excel
- CSV
- Database
- JSON
- XML

Ways to Achieve Parameterization in Cucumber

1. Using Placeholders ({string}, {int})

Feature File

Given User enters username "admin"

Step Definition

```
@Given("User enters username {string}")
public void enterUsername(String username) {
    System.out.println(username);
}
```

2. Using Scenario Outline and Examples

Feature File

Scenario Outline: Login Test

Given User enters "<username>" and "<password>"

Examples:

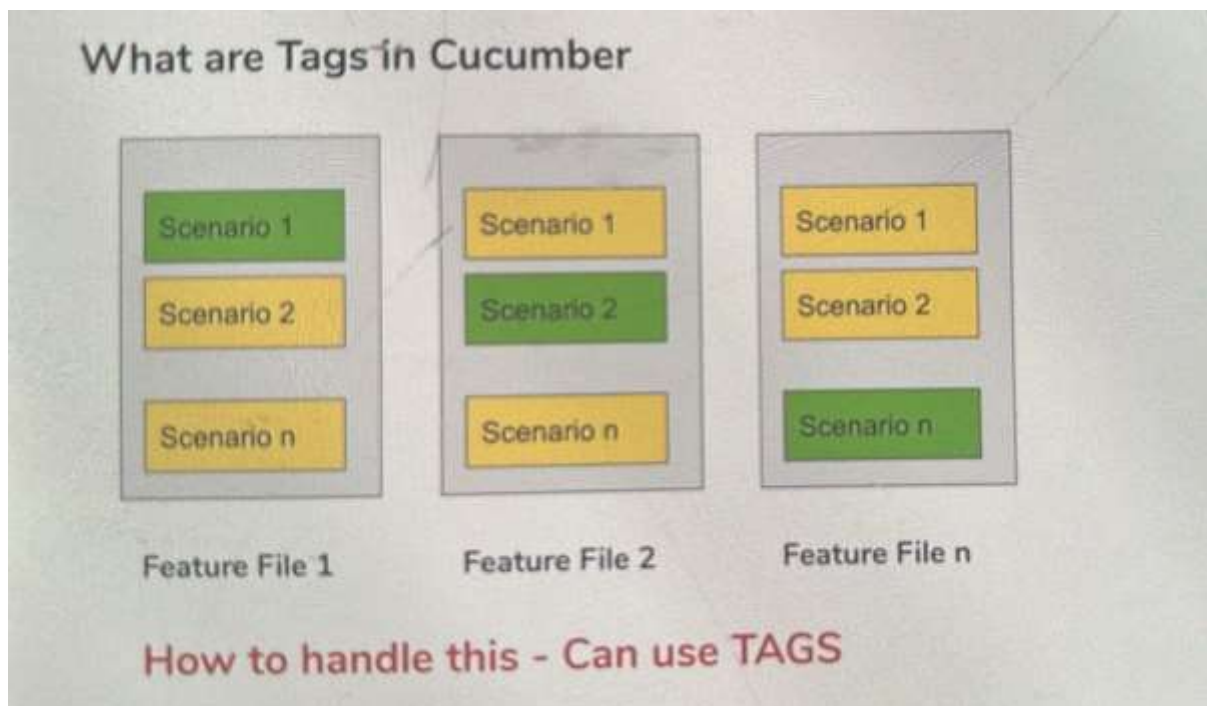
username password
admin admin123
user1 user123

The same scenario runs multiple times with different data.

Tags in Cucumber

Tags are used to group, organize, and selectively execute specific scenarios or features.

- Features and Scenarios can be marked with Tags
- Tags use @symbol with some text [e.g. @SmokeTest](#)
- A feature or scenario can have one or more tags
@smoke @regression @important
Feature: Verify login
- In the test runner class we can run specific tags
- We can run with single tag
@CucumberOptions(tags = "@Smoke")
- Can run with multiple tags or different combination of tags or using AND, OR conditions
@CucumberOptions(tags = "@Smoke or @Regression")
@CucumberOptions(tags = "@Smoke and @Regression")
Skip scenarios having a specific tag using **NOT condition**.
- @CucumberOptions(tags = "not @Regression")



Tags are used to run specific test scenarios across multiple feature files

Tags can be placed above the following **Gherkin elements**:

- Feature

- Scenario
- Scenario Outline
- Examples

It is not possible to place tags above background or steps(Given,When ,Then ,And and But)

Tags are inherited by child elements

Tags that place above a Feature can be inherited by Scenario, Scenario Outline or Examples

Hooks

"Hooks are special annotations in Cucumber that contain blocks of code that execute automatically before or after a scenario, without being explicitly mentioned in the feature file."

Common Hook Annotations

- **@Before** → Executes before every scenario.
- **@After** → Executes after every scenario.
- **@BeforeStep** → Executes before every step.
- **@AfterStep** → Executes after every step.

Why hooks?

- They are mainly used for setup and teardown activities such as launching the browser, closing the browser, establishing database connections, and capturing screenshots."
- To avoid rewriting the common setup or teardown actions
- Allow better management of code workflow

Order in Hooks

When multiple hooks of the same type are present, **Order** is used to control the execution sequence.

Example

```
@Before(order = 1)
public void setupDB() {
    System.out.println("Database Connection");
}

@Before(order = 2)
public void launchBrowser() {
    System.out.println("Browser Launch");
}
```

For @Before hooks, lower order value executes first.

For @After hooks, higher order value executes first (reverse order).

Conditional Hooks

Conditional hooks are the hooks that execute only for scenarios having specific tags

Instead of running before/after every scenario, they run only for tagged scenarios.

Example

```
@Before("@Smoke")
public void setupSmoke() {
    System.out.println("Executing Before Hook for Smoke Tests");
}
```

This hook runs only for scenarios tagged with **@Smoke**.

Feature File

```
@Smoke
Scenario: Login Test
Given User is on Login Page

@Regression
Scenario: Add Product Test
Given User is on Product Page
```

Execution

✓ Login Test → Hook executes

✗ Add Product Test → Hook does not execute

How to Use both tags and order

```
@Before(value = "@Smoke", order = 1)
public void setupDB() {
    System.out.println("Database Connection");
}
```

Background in Cucumber

Background is used to define a set of common steps that should be executed before every scenario in a feature file.

Why do we use Background?

- To avoid repeating common steps in every scenario.
- To improve readability and maintainability.
- To reduce duplication in feature files.
- Unlike hooks, background is visible to the readers of the feature file

Without Background

Scenario: Valid Login
Given User is on Login Page
When User enters valid credentials
Then Login should be successful

Scenario: Invalid Login
Given User is on Login Page
When User enters invalid credentials
Then Error message should be displayed

Here, "**Given User is on Login Page**" is repeated.

With Background

Feature: Login Functionality

Background:
Given User is on Login Page

Scenario: Valid Login
When User enters valid credentials
Then Login should be successful

Scenario: Invalid Login
When User enters invalid credentials
Then Error message should be displayed

Now the common step is written only once.

Execution Flow

Background

↓

Scenario 1

Background

↓

Scenario 2

Background

↓

Scenario 3

Background executes before every scenario.

U can only have one set of Background steps per feature

Background vs Hooks

Background

Written in **Feature File**.

Used for **common preconditions**.

Hooks

Written **Java Step Definition/Hook Class**.

Used for **setup and teardown activities**.

Background	Hooks
Visible to business users.	Not visible in the feature file.
Written using Gherkin steps (Given, When, Then).	Written using annotations like @Before, @After.
Executes before every scenario in that feature file.	Executes before/after every scenario (or tagged scenarios).
Example: User is on Login Page.	Example: Launch Browser, Close Browser, DB Connection.

One-Line Difference

☞ **"Background is for common test steps, whereas Hooks are for common automation code."** ✓

Command Line Execution

Command Line Execution means running tests or features from command line or terminal without using the IDE or GUI.

Why do we use Command Line Execution?

- Execute tests without IDE.
- Useful in Jenkins and CI/CD pipelines.
- Faster automation execution.
- Easy integration with Maven

When to Use Command Line Execution?

- Whenever you need to run the tests **without opening the IDE**.
- Whenever you need to do **integration with other processes like CI/CD and DevOps**.
- Whenever you are using **CI tools like Jenkins**.
- Whenever you need **batch execution or scheduled execution**.
- Whenever you are done with your **test creation and test setup**.

How to Perform Command Line Execution?

Step 1: Open Command Prompt / Terminal

Navigate to your project folder:

```
cd C:\Users\vara\AutomationProject
```

Step 2: Verify Maven Installation

```
mvn -version
```

Step 3: Execute All Tests

```
mvn test
```

Step 4: Execute Specific Tagged Tests

Run Smoke Tests:

```
mvn test -Dcucumber.filter.tags="@Smoke"
```

Run Regression Tests:

```
mvn test -Dcucumber.filter.tags="@Regression"
```

Run Multiple Tags:

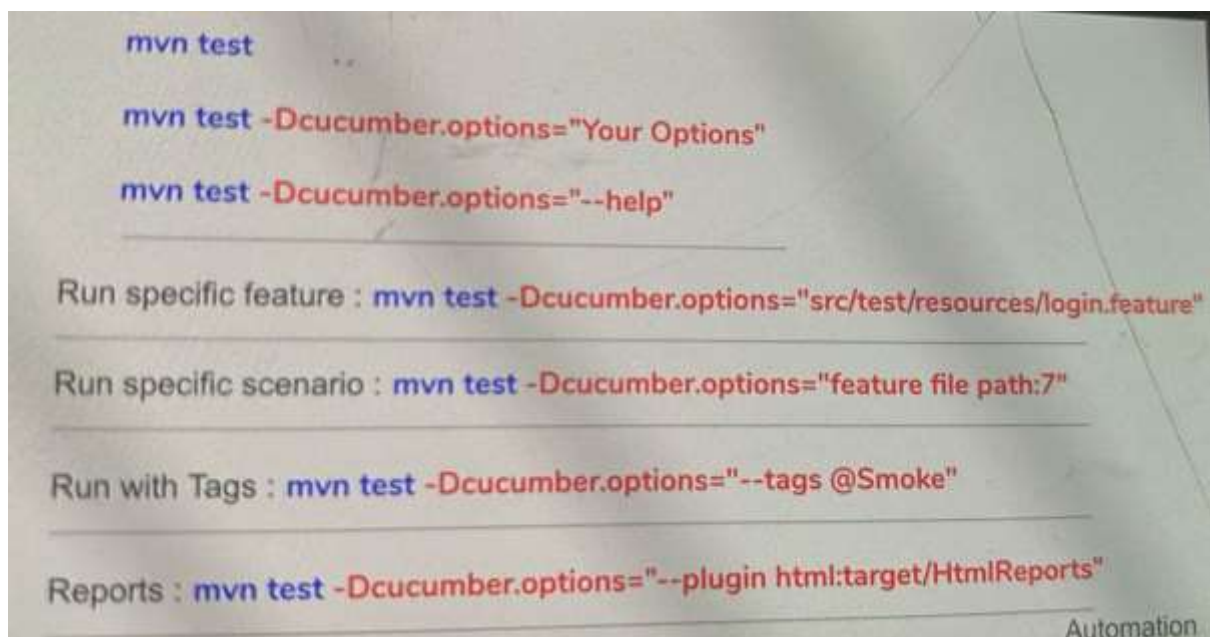
```
mvn test -Dcucumber.filter.tags="@Smoke or @Regression"
```

Step 5: Clean and Execute

```
mvn clean test
```

- `clean` → Removes previous build files.
- `test` → Executes test cases.

Old versions



Latest Versions

To Run a Specific Feature File

```
mvn test -Dcucumber.features="src/test/resources/features/Login.feature"
```

Run Specific Scenario Using Line Number

```
mvn test -Dcucumber.options="src/test/resources/features/Login.feature:7"
```

- `Login.feature` → Feature file path.
- `:7` → Line number where the scenario starts

(or)

Using tag:

```
mvn test -Dcucumber.filter.tags="@LoginTest"
```

Run with tags

```
mvn test -Dcucumber.filter.tags="@Smoke"
```

Run Multiple Tags

```
mvn test -Dcucumber.filter.tags="@Smoke or @Regression"
```

Generate Reports

HTML Report

```
mvn test -Dcucumber.plugin="html:target/cucumber-report.html"
```

JSON Report

```
mvn test -Dcucumber.plugin="json:target/report.json"
```

Unit XML Report

```
mvn test -Dcucumber.plugin="junit:target/report.xml"
```

Pretty Console Output

```
mvn test -Dcucumber.plugin="pretty"
```

Multiple Plugins

```
mvn test -  
Dcucumber.plugin="pretty,html:target/report.html,json:target/report.json"
```

In old version

Running with multiple parameters

Can run with multiple parameters from command line

```
mvn test -Dcucumber.options="-----" -Dcucumber.options="-----"
```

example

```
mvn test  
-Dcucumber.options="src/test/resources/login.feature"  
-Dcucumber.options="--tags @Smoke"
```

In new version

Run Specific Feature + Tags

```
mvn test -Dcucumber.features="src/test/resources/login.feature" -Dcucumber.filter.tags="@Smoke  
or @Regression"
```

By default mvn test will run the files with naming syntax(Test Runner file)

```
**/Test*.java
```

```
**/*Test.java
```

```
**/*TestCase.java
```

Reports

"Reports in Cucumber are used to capture and display test execution results in different formats such as HTML, JSON, XML, and Console output."

Why do we use Reports?

- To view test execution results.
- To identify passed, failed, and skipped scenarios.
- To share execution results with stakeholders.
- To integrate with CI/CD tools like Jenkins.
- To analyze test execution history.

Dry Run

```
@CucumberOptions(dryRun = true)
```

"Dry Run is used to check whether all feature file steps have corresponding step definitions without executing the tests."

Interview Questions and answers

What is Gherkin?

Answer:

Gherkin is the language used to write Cucumber feature files. It uses keywords such as Feature, Scenario, Given, When, Then, And, and But.

Why do we use Cucumber?

"Cucumber improves collaboration between developers, testers, and business teams by allowing test cases to be written in a readable format."

What are Plugins?

"Plugins are used to generate execution reports and customize output."

Examples:

- pretty
- html
- json
- junit

Why Selenium + Cucumber?

"Selenium is used for browser automation, while Cucumber provides a BDD framework to write test scenarios in a readable format. Together they enable business-readable automation testing."

What is a Feature File?

A Feature File contains multiple scenarios related to a particular feature and is written in Gherkin language with a `.feature` extension."

. What is a Scenario?

A Scenario is a single test case inside a Feature File that validates a specific functionality."

What is Scenario Outline?

Answer:

Scenario Outline is used to execute the same scenario multiple times with different sets of test data using an Examples table.

Difference Between Scenario and Scenario Outline?

Scenario

- Executes once.
- Uses one set of data.

Scenario Outline

- Executes multiple times.
- Uses multiple sets of data through Examples table.

. What is a Step Definition?

Answer:

A Step Definition contains Java code that implements the steps written in the feature file.

What is a Test Runner Class?

Answer:

A Test Runner Class is used to execute Cucumber feature files and configure options such as features, glue, tags, and plugins.

What is Glue?

Answer:

Glue specifies the package location of Step Definition and Hook classes.

```
glue="stepDefinitions"
```

How Do You Run a Specific Scenario?

Answer:

Using tag:

```
mvn test -Dcucumber.filter.tags="@LoginTest"
```

Using line number (older versions):

```
mvn test -Dcucumber.options="Login.feature:10"
```

Difference Between Background and Scenario Outline

Background

- Common steps before every scenario.

Scenario Outline

- Same scenario with multiple data sets.

Cucumber Architecture

Feature File
↓
Step Definition
↓
Runner Class
↓
Selenium WebDriver
↓
Application

Difference Between And and But

And

- Additional positive condition.

But

- Additional negative condition.

What is Monochrome in Cucumber?

"Monochrome is an option in Cucumber that makes the console output more readable and cleaner by removing unnecessary special characters and formatting."

Syntax

```
@CucumberOptions(  
    monochrome = true  
)
```